

Lab 8: Strings (with Pointers)

Computer Programming (010153002) | Semester 1/2026

Duration: 2 hours (in-lab) | **Topic:** Null-terminated `char` arrays, `char *` traversal, `string.h`

Objectives

- Represent strings as null-terminated `char` arrays.
- Use both indexing (`s[i]`) and pointer traversal (`*p, p++`) on strings.
- Read strings safely with `scanf("%s", ...)` and `fgets`.
- Use `strlen`, `strcpy`, `strcmp`, `strcat`, and understand why `a == b` does not compare strings.

Quick Reference

A C string is a `char` array ending in `'\0'`.

```
char s[20] = "hello"; /* 6 bytes: 'h','e','l','l','o','\0' */
```

Two equivalent traversals.

```
/* index style */          /* pointer style */
for (int i = 0; s[i]; i++) ... for (char *p = s; *p; p++) ...
```

Both stop at the null terminator. They produce identical results because $s[i] \equiv *(s + i)$.

Reading input. `scanf("%s", s);` reads one whitespace-delimited word and writes `'\0'`.

`fgets(s, sizeof s, stdin);` reads a whole line including `'\n'`.

Library functions (`<string.h>`):

<code>strlen(s)</code>	number of chars before the <code>'\0'</code>
<code>strcpy(dst, src)</code>	copy (dst must be large enough)
<code>strcmp(a, b)</code>	0 if equal, < 0 if <code>a < b</code> , > 0 otherwise
<code>strcat(dst, src)</code>	append <code>src</code> onto <code>dst</code>

Pitfall. `a == b` compares *pointers*, not contents. Use `strcmp`.

Quick Experiments (Try It)

Try It 1: String comparison with `==` vs. `strcmp`

Run and study the results:

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char a[] = "hello";
    char b[] = "hello";
    printf("a == b (pointer): %d\n", a == b);
    printf("strcmp(a,b): %d\n", strcmp(a, b));
    printf("address of a: %p\n", (void *)a);
    printf("address of b: %p\n", (void *)b);
    return 0;
}
```

1. Why does `a == b` print 0 (false) even though both arrays contain "hello"?
2. What are `a` and `b` when used in `==`? (Look at the printed addresses.)
3. When does `strcmp` return 0, a positive value, or a negative value?

Try It 2: The hidden null terminator and strlen vs. sizeof

Run and explain each line:

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char s[10] = "hello";
    printf("strlen: %lu\n", (unsigned long)strlen(s));
    printf("sizeof: %lu\n", (unsigned long)sizeof(s));
    s[2] = '\0';           /* plant a null in the middle */
    printf("strlen after: %lu\n", (unsigned long)strlen(s));
    printf("string after: %s\n", s);
    return 0;
}
```

1. What is the difference between `strlen` and `sizeof` for arrays?
2. After `s[2] = '\0'`, what does `%s` print and why?
3. Where are the characters 'l', 'l', 'o' in memory? Are they still there? How could you print them?

Try It 3: Index style vs. pointer style – they do the exact same thing

Add print statements to verify both traversals visit identical characters:

```
#include <stdio.h>
int main(void) {
    char s[] = "World";
    /* index style */
    for (int i = 0; s[i] != '\0'; i++)
        printf("idx[%d] = %c (%d)\n", i, s[i], s[i]);
    /* pointer style */
    for (char *p = s; *p != '\0'; p++)
        printf("ptr = %c (%d) addr=%p\n", *p, *p, (void*)p);
    return 0;
}
```

1. Do both loops print the same characters in the same order?
2. How much does the pointer address increase with each iteration?
3. Rewrite the pointer loop to use `*(s + i)` with an index variable. Does it still work identically?

In-Lab Exercises (target: 2 hours)**In-Lab Exercise 1: my_strlen – index and pointer versions (25 min)**

Write two functions that compute the length of a string *without* calling `strlen`:

- `int len_idx(const char s[])` – uses `s[i]` and an index.
- `int len_ptr(const char *s)` – walks a pointer `p` until `*p == '\0'`.

Verify both agree with `strlen` for at least three test inputs.

In-Lab Exercise 2: Reverse a String In Place (two pointers) (30 min)

Write `void reverse(char *s)` that reverses a string in place using two pointers (one starting at the beginning, one at the character before the `'\0'`). Test with input read via `scanf("%s", ...)`.

In-Lab Exercise 3: Count Vowels and Consonants (25 min)

Read a line with `fgets` and count vowels (case-insensitive), consonants, and other characters. Use pointer traversal: `for (char *p = line; *p; p++) ...`

In-Lab Exercise 4: Palindrome with Two Pointers (40 min)

Write `int is_palindrome(const char *s)` that returns 1 if `s` reads the same forward and backward (case-insensitive, ignoring spaces). Use two pointers `left` and `right` moving toward the middle. Test on "radar", "A man a plan a canal Panama", and "hello".

AI Collaboration

Try these prompts with an AI tool:

- “Explain why `if (a == b)` does not compare two C strings, and what to use instead.”
- “Show me how to implement `strlen` manually using pointer traversal, without calling the library version.”
- “What is a buffer overflow in the context of `strcpy`, and how does `strncpy` reduce (but not eliminate) the risk?”

Critical check – verify, don’t just accept: Ask the AI to write a palindrome-checking function that handles spaces and mixed case. Before running, inspect the AI’s loop termination condition and predict whether it handles an empty string "" without reading out of bounds. Test on: "radar", "A man a plan a canal Panama", "hello", and "". Record whether the AI’s stated behavior matches the actual output.

Know the limit: AI tools frequently write C string code that works for typical inputs but silently overflows a fixed-size buffer on longer input – always ask “what happens if the input is longer than the array?” as a follow-up.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Null terminator inside the buffer

What does this print?

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char s[10] = {'C', 'S', '\0', '1', '0', '1', '\0'};
    printf("%lu\n", (unsigned long)strlen(s));
    printf("%s\n", s);
    return 0;
}
```

Trace & Predict 2: Pointer-walk equivalence

Predict the output of both loops:

```
#include <stdio.h>
```

```
int main(void) {
    char s[] = "abc";
    for (int i = 0; s[i]; i++)        printf("%c", s[i]);
    printf(" / ");
    for (char *p = s; *p; p++)        printf("%c", *p);
    printf("\n");
    return 0;
}
```

Find the Bug 1: Comparing strings with == and buffer too small

List every bug and rewrite the corrected program.

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char a[3];
    char b[] = "yes";
    scanf("%s", a);
    if (a == b) printf("match\n");
    else       printf("no match\n");
    return 0;
}
```

Write Code on Paper 1: Uppercase via pointer

Write `void to_upper(char *s)` that walks the string with a pointer and converts every lowercase letter to uppercase. Do not use `ctype.h`. Then write a `main` that reads a word and prints it uppercased.

Write Code on Paper 2: Count occurrences of a character

Write `int count_char(const char *s, char c)` that returns the number of times `c` occurs in `s` (use pointer traversal). Demonstrate from `main` on input read with `fgets`.

Draw on Paper 1: String memory layout

Part A. Draw the memory layout of `char s[] = "Hello"` as a row of labelled boxes. For each cell show: the **index**, the **character**, and its **decimal ASCII value**.

Part B. Draw the layout of `char t[7] = {'H', 'i', '\0', '!', '?', '\0', '\0'}`. Mark with an arrow where `strlen(t)` stops counting, and write the value it returns.